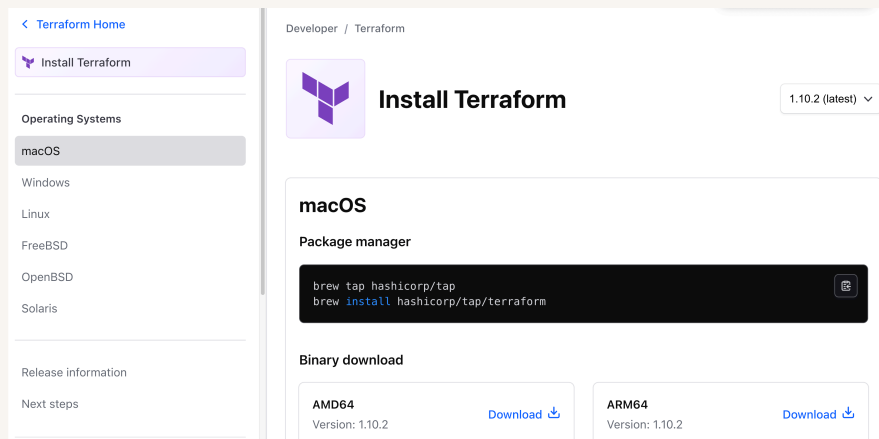




Create S3 Buckets with Terraform



Natasha Ong





Introducing Today's Project!

In this project, we will demonstrate how to use Terraform to launch an S3 bucket. The goal is to install + set up Terraform, troubleshoot any errors (i.e. installing the CLI), and then successfully apply our Terraform configuration to launch a bucket.

Tools and concepts

Services we used were Amazon S3, AWS IAM, Terraform and AWS CLI. Key concepts we learnt include infrastructure as code, creating configuration files, modularity of code, using providers, plugins, state files, lock files (Terraform concepts)

Project reflection

This project took us approximately 1.5 hours including demo time. The most challenging part was customizing our S3 configuration using the official Terraform documentation. It was most rewarding to see a local image uploaded into the S3 bucket.

We did this project because we are complete beginners to Terraform and wanted to make a start. This project met our goals because we could actually install + set up Terraform, and use it to launch resources in our AWS environment.

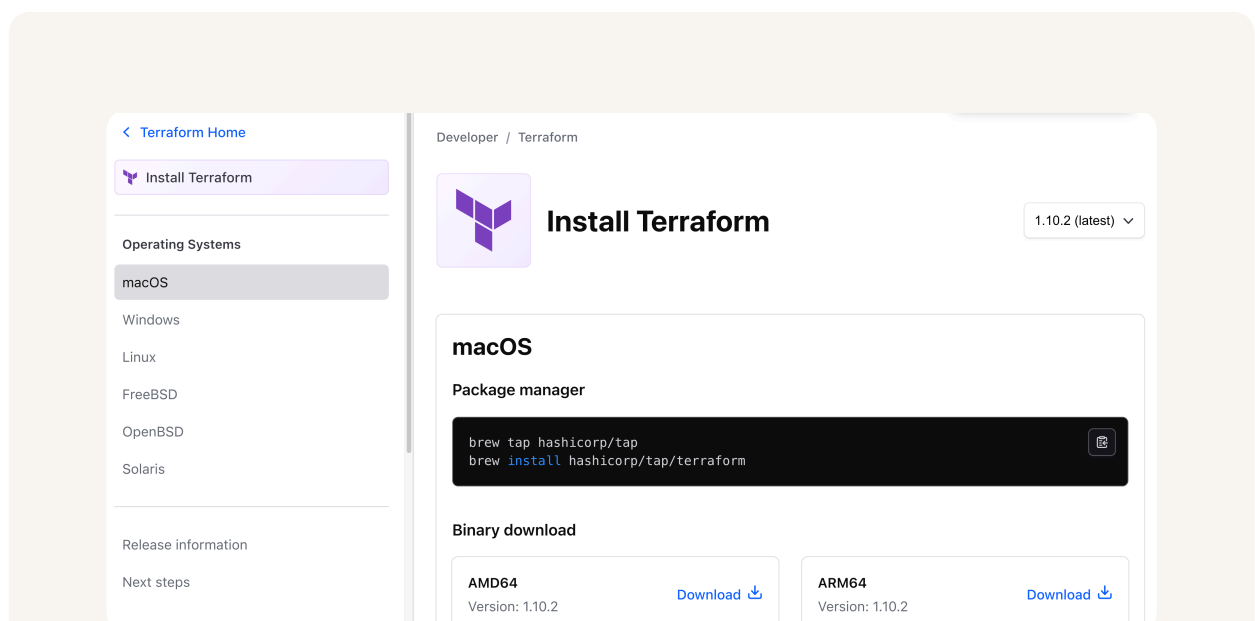


Introducing Terraform

Terraform is a tool for managing our IT resources using code. You use Terraform to process a configuration file you've prepared that details the desired state of your infrastructure. Terraform then creates/updates/delete to set up that desired state.

Terraform is one of the most popular tools used for infrastructure as code (IaC), which is a way to manage your IT infrastructure. Instead of manually managing resources using the console / text commands in CLI, IaC automates that process with code.

Terraform uses configuration files to understand the desired state of our infrastructure. `main.tf` is the main configuration file that we use in Terraform to describe that desired state.





Configuration files

The configuration is structured in blocks instead of a single block of code. The advantage of doing this (i.e. modularity) is the ability to work on/update different parts of main.tf without affecting other parts. Great for teams collaborating too.

My main.tf configuration has three blocks

The first block indicates that we are using AWS as our provider for this infrastructure. The second block provisions an Amazon S3 bucket and gives it a unique name. The third block manages the bucket's permission i.e. blocking all public access.

```
main.tf
Users > natashaong > Desktop > nextwork_terraform > main.tf
1  provider "aws" {
2      region = "us-east-1" # Update this to the Region closest to you
3  }
4
5  resource "aws_s3_bucket" "my_bucket" {
6      bucket = "nextwork-unique-bucket-nextwork-1234" # Ensure this bucket name is globally unique
7  }
8
9  resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
10     bucket = aws_s3_bucket.my_bucket.id
11
12     block_public_acls       = true
13     ignore_public_acls     = true
14     block_public_policy    = true
15     restrict_public_buckets = true
16 }
```




Customizing my S3 Bucket

For our project extension, we visited the official Terraform documentation to look for ways we can customize our bucket's configurations. The documentation shows example configurations, all the available parameters for a resource and usage rules.

I chose to customise my bucket by adding tags, because that lets me identify the project that I launched it for later on. When I launch my bucket, I can verify my customization by visiting the 'Tags' panel in the S3 console for this bucket.

```
resource "aws_s3_bucket" "my_bucket" {  
  bucket = "nextwork-unique-bucket-nextwork-1234" # En  
  
  tags = {  
    Project = "Create an S3 Bucket with Terraform"  
  }  
}
```



Terraform commands

We ran 'terraform init' to initialize Terraform. This means getting it set up the backend to store state files, and install the necessary plugins i.e. the AWS provider plugin.

Next, we ran 'terraform plan' to get Terraform to compare the infrastructure/resources in main.tf against our infrastructure's current state. Then, share back the execution plan i.e. how running main.tf will impact our infrastructure e.g. create.

```
NextWork: $ terraform init
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

```
- Finding latest version of hashicorp/aws...
```

```
- Installing hashicorp/aws v5.81.0...
```

```
- Installed hashicorp/aws v5.81.0 (signed by HashiCorp)
```

```
Terraform has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that Terraform can guarantee to make the same selections by default when  
you run "terraform init" in the future.
```

```
Terraform has been successfully initialized!
```

```
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.
```

```
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.
```



AWS CLI and Access Keys

When we tried to plan our Terraform configuration, we received an error message that says 'no valid credentials found', because our local terminal wasn't set up with AWS credentials yet. This needs to be set up to use the AWS provider plugin.

To resolve our error, first we installed AWS CLI, which is AWS' command line tool for interacting with our AWS infrastructure. Instead of using the Management Console, the CLI lets us run text commands to do the same things from our local computer.

We set up AWS access keys so that we have a way to 'log in' to our AWS account over the CLI in our local computer. Once the Access Key ID and Secret Access Key are passed through 'aws configure' our terminal now has the necessary AWS credentials.

```
NextWork: $ terraform plan
Planning failed. Terraform encountered an error while generating this plan.

Error: No valid credential sources found

with provider["registry.terraform.io/hashicorp/aws"],
on main.tf line 1, in provider "aws":
   1: provider "aws" {

Please see
https://registry.terraform.io/providers/hashicorp/aws
for more information about providing credentials.

Error: failed to refresh cached credentials, no EC2 IMDS
role found, operation error ec2imds: GetMetadata,
exceeded maximum number of attempts, 3, request send
failed, Get
"http://169.254.169.254/latest/meta-data/iam/security-credentials/":
dial tcp 169.254.169.254:80: connect: host is down
```



Lanching the S3 Bucket

We ran 'terraform apply' to get Terraform to apply the changes/updates it showed us when we ran 'terraform plan'. Running 'terraform apply' will affect my AWS account by creating two resources - an S3 bucket, and its set of permission settings we.

The sequence of running terraform init, plan and apply is crucial because we need to initialize Terraform first before we get to (1) make any comparisons between main.tf and our current infrastructure, or (2) connect to AWS in the first place.

 [Amazon S3](#) > [Buckets](#) > nextwork-unique-bucket-nextwork-1234

Tags (1)

You can use bucket tags to track storage costs and organize buckets. [Learn more](#) 

Key	Value
Project	Create an S3 Bucket with Terraform



Uploading an S3 Object

We created a new resource block to upload an image called 'image.png' into the S3 bucket. The image will be uploaded straight away without being nested in a subfolder in the bucket.

We need to run terraform apply again we've updated our Terraform configuration file, which means there are changes that Terraform needs to review and compare with our current infrastructure again. This time, only one new resource is created (image).

To validate that we've updated our configuration successfully, we checked the S3 bucket and confirmed that a new image is inside. We also downloaded the image back to us, and confirmed that it was the correct image that was initially uploaded.

```
22 resource "aws_s3_object" "image" {
23     bucket = aws_s3_bucket.my_bucket.id # Reference the bucket ID
24     key    = "image.png" # Path in the bucket
25     source = "image.png" # Local file path
26 }
27 |
```



NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

